



Retrieval Augmented Generation with Gemini 2 and Automatic Evaluation

NLP

École Centrale de Casablanca

Academic Year 2024/2025

Réalisé par :
Hamza El Otmani
Reda Benkirane
Adam Lozi
Ayoub Slimani

March 15, 2025

Contents

1	Introduction	3
2	Background and Related Work	3
3	System Architecture	3
3.1	Model Selection	3
3.2	Document Processing Pipeline	3
3.3	Chat Engine	3
4	Implementation Details	3
4.1	Document Handling	4
4.2	Model Interaction	4
4.3	Response Evaluation	4
5	Challenges and Limitations	4
5.1	API Constraints	4
5.2	Vector Search Accuracy	4
5.3	Response Consistency	4
5.4	Scalability	4
5.5	User Adaptability	4
6	Future Work and Enhancements	5
6.1	Enhanced Embedding Models	5
6.2	Optimized Retrieval Mechanisms	5
6.3	System Scalability	5
6.4	User-Centric Improvements	5
7	Conclusion	5

1 Introduction

Retrieving relevant knowledge from large documents has become challenging. Traditional keyword-based search engines often don't work well when dealing with complex, context-dependent queries. To bridge this gap, retrieval-augmented generation (RAG) techniques Lewis et al. [2021] have emerged as a powerful solution, combining the strengths of information retrieval and natural language generation. This project aims to enhance document-based question answering by using state-of-the-art Google's Gemini models like *Gemini 2* Research [2023] and also using *LlamaIndex* Developers [2022] to enable intelligent, context-aware interactions.

The objective of this application is to provide an interactive, user-friendly chat interface capable of processing and retrieving information from multiple documents efficiently. By integrating *LlamaIndex*, the system can preprocess documents, extract embeddings, and structure information for efficient retrieval. *Gemini* models then refine and generate responses, ensuring that answers are both contextually relevant and coherent.

2 Background and Related Work

Advancements in NLP and Retrieval-Augmented Generation (RAG) Lewis et al. [2021] have significantly improved AI applications. Traditional chatbots rely on static training data, limiting their ability to handle dynamic queries. RAG models enhance this by integrating external knowledge retrieval with generative AI, enabling more accurate and context-aware responses.

Prior work on RAG systems Lewis et al. [2021] has demonstrated improved factual consistency by retrieving relevant context before generation. Additionally, *LlamaIndex* Developers [2022] optimizes indexing and retrieval, reducing hallucinations in large language models.

3 System Architecture

The *RAG application* is designed as an interactive chatbot leveraging Retrieval-Augmented Generation for enhanced response accuracy. It consists of three core components: model selection, document processing, and the chat engine.

3.1 Model Selection

We utilize Google's Gemini 1.5 Pro and Gemini 2.0 Flash, chosen for their efficiency in multi-turn dialogue and retrieval-based augmentation. These models Research [2023] balance computational cost with performance, ensuring responsive and contextually relevant interactions.

3.2 Document Processing Pipeline

Documents are ingested, chunked into manageable sections, embedded using a vectorization model, and stored in a vector database. *LlamaIndex* Developers [2022] facilitates efficient indexing and retrieval, optimizing the chatbot's ability to access relevant context dynamically.

3.3 Chat Engine

The chat interface, built with Streamlit, handles user queries by retrieving relevant document chunks before passing them to the Gemini models. The response generation pipeline integrates retrieval scoring and filtering mechanisms to improve relevance and reduce hallucinations.

4 Implementation Details

The implementation of is structured around three key modules: document handling, model interaction, and response evaluation.

4.1 Document Handling

Documents are preprocessed by tokenizing, chunking, and embedding into a vector store. The indexing mechanism ensures efficient retrieval by aligning stored text with user queries, enhancing response relevance.

4.2 Model Interaction

LlamaIndex orchestrates the retrieval and augmentation process. User queries are vectorized and matched against indexed documents before being sent to the Gemini model, ensuring responses incorporate relevant contextual information.

4.3 Response Evaluation

A high quality assessment mechanism scores responses based on relevance, coherence, and factual consistency. Retrieval confidence and filtering techniques minimize hallucinations and improve chatbot reliability. This method is called *Automatic Evaluation*.

5 Challenges and Limitations

5.1 API Constraints

The reliance on external APIs introduces rate limits and potential downtime, affecting response latency and availability. Caching frequently used embeddings and optimizing API calls mitigate these issues but do not eliminate dependency risks.

5.2 Vector Search Accuracy

The effectiveness of retrieval depends on embedding quality and search algorithms. Poorly indexed documents or non-optimal similarity metrics can lead to irrelevant responses. Fine-tuning chunking strategies and experimenting with different embedding models help improve precision.

5.3 Response Consistency

Large language models may generate inconsistent or factually incorrect responses (hallucinations). While retrieval-augmented generation (RAG) reduces hallucinations, the system still requires post-processing techniques, such as response validation and confidence scoring, to enhance reliability.

5.4 Scalability

Handling large document sets increases memory and computational overhead. Efficient indexing, sharding techniques, and distributed processing strategies help maintain system performance under high workloads.

5.5 User Adaptability

The application's effectiveness depends on user query formulation. Poorly structured inputs may lead to suboptimal responses. Interactive refinements, feedback loops, and prompt engineering techniques can improve adaptability.

6 Future Work and Enhancements

6.1 Enhanced Embedding Models

Future iterations can integrate advanced embedding models to improve retrieval accuracy. Exploring domain-specific embeddings or fine-tuning existing models could further refine document representation.

6.2 Optimized Retrieval Mechanisms

Improving the retrieval pipeline by experimenting with hybrid search techniques, such as dense-sparse fusion et al. [2022], can enhance response relevance. Adaptive ranking methods based on user feedback could further refine retrieval quality.

6.3 System Scalability

Optimizing the indexing process for large-scale datasets using distributed vector search frameworks can enhance scalability. Exploring approximate nearest neighbor (ANN) techniques may reduce retrieval latency.

6.4 User-Centric Improvements

Incorporating interactive query refinement, real-time feedback mechanisms, and customizable retrieval settings can enhance user experience and adaptability to different use cases.

7 Conclusion

To conclude, this is an interactive RAG application leveraging Gemini models and LlamaIndex for efficient document retrieval. By integrating embeddings and retrieval techniques, the system improves information access and response quality. Future improvements will focus on scalability, multi-modal integration, and user-centric enhancements to further refine performance and usability.

References

- LlamaIndex Developers. Llamaindex: Optimizing information retrieval for large language models, 2022. URL <https://example.com/llamaindex>.
- X. Gao et al. Dense-sparse fusion for improved document retrieval. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1025–1034, 2022. doi: 10.1145/3477495.3531849.
- Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Proceedings of the 38th International Conference on Machine Learning*, 2021. URL <https://arxiv.org/abs/2005.11401>.
- Google Research. Gemini 2 models for nlp tasks, 2023. URL <https://example.com/gemini2>.